

嵌入式面试八股文

STM32 / MCU / FreeRTOS

中小厂实习面试版

A4 打印背诵版

目 录

1. 电平与通信协议	3
1.1 常见电平	3
1.2 I2C / IIC	3
1.3 SPI	4
1.4 UART / USART	5
1.5 UART / I2C / SPI 对比	7
1.6 CAN	7
1.7 RS232 / RS485	8
1.8 Modbus RTU / RS485	9
1.9 MQTT	10
2. MCU 与 STM32	11
2.1 启动与内核	11
2.2 中断与 NVIC	11
2.3 GPIO	12
2.4 时钟与定时器	13
2.5 ADC / DAC	14
2.6 DMA	14
2.7 看门狗、Flash 与 Bootloader	14
2.8 HAL、LL、寄存器与调试	15
3. C 语言高频	16
3.1 编译、链接与存储	16
3.2 指针与数组	17
3.3 宏、头文件与类型	19
3.4 内存与字符串	20
3.5 结构体、联合体与对齐	21
3.6 数据结构与工程写法	21
4. FreeRTOS	23
4.1 任务与调度	23
4.2 SysTick / PendSV / SVC	23
4.3 队列、信号量、互斥量、事件组、通知	24
4.4 ISR 与临界区	25
4.5 内存与任务栈	25
5. Linux 基础（MCU 岗够用版）	27
6. 项目追问：把“会背”变成“会做”	28
7. 面试前 10 分钟速记表	30

1. 电平与通信协议

1.1 常见电平

□□□ [常考] TTL 电平和 CMOS 电平能不能背一个固定阈值？

不建议。逻辑阈值最终以器件 **Datasheet** 为准。经典 5V TTL 常见输入阈值约为 $V_{IL} \leq 0.8V$ 、 $V_{IH} \geq 2.0V$ ；输出指标常见 $V_{OL} \leq 0.4V$ 、 $V_{OH} \geq 2.4V$ 。MCU 的 3.3V GPIO 多为 CMOS 阈值，不能直接套 TTL 数字。

面试注意：面试中如果被问“3.3V 一定能驱动 5V TTL 吗”，要回答“看接收端 V_{IH} 和发送端 V_{OH} ，不能只看标称电压”。

□□□ [常考] RS232 电平有什么特点？

RS232 使用正负电压并且逻辑极性与 TTL UART 常见表示相反。典型接收判定：负电压表示逻辑 1，正电压表示逻辑 0；MCU UART 不能直接接 RS232，需要 MAX3232 一类电平转换器。

1.2 I2C / IIC

□□□ [高频] I2C 为什么要加上拉电阻？

- SDA、SCL 在标准 I2C 中使用开漏/开集电极结构，器件主动拉低，但不能主动输出高电平。
- 上拉电阻负责在所有器件释放总线时把线路拉到高电平，形成明确的空闲状态。
- 开漏 + 上拉允许多个器件共享线路并实现“线与”，避免一个输出高、另一个输出低造成硬短路。
- 上拉阻值会影响上升时间、灌电流和功耗，4.7kΩ 只是常见起点，不是固定答案。

□□□ [高频] I2C 为什么用开漏输出？

- 实现多器件共享总线和无损仲裁：任一设备拉低，总线就是低。
- 支持时钟同步和 clock stretching：目标设备可以拉低 SCL 延长低电平。
- 便于做电平转换，但上拉电压必须满足所有器件的电气规格，不能简单“上拉到最高电压”。

□□□ [高频] I2C 有几根线？空闲电平是什么？

两根：SCL 时钟线、SDA 数据线。标准总线空闲时两根线都为高电平。

□□□ [常考] I2C 7 位地址有多少个常规可用地址？

7 位理论空间是 128 个，但 0x00-0x07 与 0x78-0x7F 属于保留/特殊用途范围。常规 7 位目标地址通常使用 0x08-0x77，共 112 个。总线上发送地址阶段时，会形成“7 位地址 + 1 位 R/W”。

□□□ [常考] I2C 常见速率有哪些？

- Standard-mode: 100 kbit/s。
- Fast-mode: 400 kbit/s。
- Fast-mode Plus: 1 Mbit/s。
- High-speed mode: 最高 3.4 Mbit/s，要求器件支持。

□□□ [高频] I2C 的 START、STOP、数据有效条件是什么？

- **START**: SCL 为高时, SDA 从高变低。
- **STOP**: SCL 为高时, SDA 从低变高。
- 正常数据传输期间, SCL 高电平时 SDA 应保持稳定; SDA 通常在 SCL 低电平期间改变。
- 每个字节按 MSB first 传输, 随后有第 9 个 ACK/NACK 时钟。

□□□ [高频] ACK 和 NACK 分别表示什么？

ACK 是接收方在第 9 个时钟把 SDA 拉低。NACK 是释放 SDA 为高。NACK 不一定表示错误: 主机作为接收方读取最后一个字节后, 会发送 NACK 告诉从设备“读取结束”, 再发 STOP。

□□□ [进阶] I2C 上拉电阻怎么估算？

需要同时满足“不能太大”和“不能太小”。常用上界近似 $R_p(\max) = t_r / (0.8473 * C_b)$, 保证上升时间; 下界近似 $R_p(\min) = (V_{DD} - V_{OL}(\max)) / I_{OL}$, 保证器件拉低时电流不超规格。实际还要结合总线电容、线长、速率和器件能力。

□□□ [常考] 硬件 I2C 和软件 I2C 有什么区别？

- **硬件 I2C**: 由外设产生时序, CPU 占用低、通常支持中断/DMA, 时序更容易满足规范。
- **软件 I2C**: GPIO bit-bang, 管脚选择灵活、调试直观, 但占 CPU, 速度和实时性更依赖软件实现。
- 不能绝对说“软件 I2C 一定不稳定”; 稳定性取决于实现、时序和系统负载。

□□□ [常考] I2C 多主机怎么仲裁？

控制器在发送 SDA 的同时读取总线。若自己释放 SDA 想发 1, 却读到总线为 0, 说明其他控制器发送了显性低电平, 当前控制器失去仲裁并停止争用。整个过程不会破坏胜出者的数据。

□□□ [常考] 什么是 Clock Stretching？

目标设备暂时处理不过来时, 可以保持 SCL 为低, 延长时钟低电平时间; 控制器必须读取实际 SCL 电平, 等它真的回到高电平时再继续。前提是控制器和目标设备都支持该机制。

□□□ [项目高频] I2C 总线死锁怎么处理？

- 先判断 SDA/SCL 是否被长期拉低。
- 如果 SDA 被从设备卡住, 可尝试让主机产生若干 SCL 脉冲(工程中常用 9 个或更多, 按器件手册), 让从设备完成未结束的字节。
- 随后产生 STOP, 再重新初始化 I2C 外设。
- 仍无效时考虑硬件复位/掉电重启目标器件, 并排查上拉、电源、线长和噪声。

1.3 SPI

□□□ [高频] SPI 的基本特点是什么？

SPI 是同步串行接口, 常见为主从结构, 通常支持全双工, 速率可以很高。典型信号为 SCLK、MOSI、MISO、CS/SS。

□□□ [高频] SPI 四种模式怎么区分？

- CPOL 决定 SCLK 空闲电平: 0 表示空闲低, 1 表示空闲高。

- **CPHA** 决定在第一个还是第二个有效边沿采样。
- $\text{Mode0}=\text{CPOL0/CPHA0}$; $\text{Mode1}=0/1$; $\text{Mode2}=1/0$; $\text{Mode3}=1/1$ 。
- 实际使用必须与从设备 **Datasheet** 保持一致。

□□□ [高频] SPI 的四根线分别做什么？

- **SCLK**: 主机提供时钟。
- **MOSI**: Master Out, Slave In, 主机发、从机收。
- **MISO**: Master In, Slave Out, 主机收、从机发。
- **CS/SS**: 片选, 通常低有效, 用来选择从设备并划分一帧事务。

□□□ [常考] SPI 可以减少信号线吗？

可以, 但要看设备协议。只写设备时可能不接 **MISO**, 只读设备可能不需要 **MOSI**; 单从设备时也不能想当然地把 **CS** 省掉, 因为很多器件依赖 **CS** 边沿划分命令和复位内部状态机。只有 **Datasheet** 明确允许时才能固定片选。

□□□ [纠错必背] SPI 有没有统一的设备地址和读写位？

没有。**SPI** 只定义基本的时钟/数据传输方式, 不规定“7 位地址+1 位读写位”。某些传感器会把第一个字节设计成寄存器地址并在最高位放 R/W, 这是该芯片的私有协议。

□□□ [纠错必背] SPI 是不是一定 MSB first？

不是。很多器件使用 **MSB first**, 但 **SPI** 控制器和外设可能支持 **MSB first** 或 **LSB first**。以从设备协议为准, 主从必须配置一致。

□□□ [常考] SPI 的“全双工”怎么理解？

每产生一个 **SCLK**, **MOSI** 和 **MISO** 可以同时各移出一位。即使你只想“读”, 主机通常也需要发送 **dummy** 数据来产生时钟。

□□□ [了解] Dual SPI / Quad SPI 是什么？

普通 **SPI** 通常有 1 根输入和 1 根输出数据线; **Dual/Quad** 模式把多根 **IO** 线作为双向数据线并行传输。**Quad SPI** 常用于外部 **NOR Flash**, 可显著提高吞吐率; 具体命令阶段、地址阶段和数据阶段宽度看器件协议。

□□□ [项目高频] SPI 常见通信故障怎么排查？

- 先核对 **CPOL/CPHA**、位序、数据宽度和时钟频率。
- 确认 **CS** 的有效电平、建立/保持时间以及一次事务中是否被意外拉高。
- 示波器/逻辑分析仪看 **SCLK**、**MOSI**、**MISO** 是否符合 **Datasheet**。
- 确认主从共地、电平兼容、线长和驱动能力。

1.4 UART / USART

□□□ [高频] UART 最基本的线有哪些？

最常见是 **TX**、**RX**, 再加公共 **GND**。全双工时 **TX** 对 **RX** 交叉连接; 单端 **TTL UART** 通常只适合板内或短距离通信。

□□□ [高频] UART 的帧格式是什么？

空闲线通常为高电平；1 个起始位（低），若干数据位（常见 8 位，通常 LSB first），可选校验位，1/1.5/2 个停止位（高）。双方必须约定相同波特率和帧格式。

□□□ [了解] 波特率和 bit/s 是一回事吗？

在常见二进制 UART 中一个码元承载 1 bit，因此波特率数值通常等于 bit/s；但“波特率”严格说是每秒码元数。面试中说 115200 baud 即常见 UART 位速率即可。

□□□ [常考] UART 为什么没有时钟线也能通信？

发送端和接收端预先约定波特率。接收端检测起始位下降沿后，利用自身时钟在每个比特中心附近采样，因此两端时钟允许有一定误差，但误差太大会导致帧错误。

□□□ [项目高频] UART 轮询、中断、DMA 怎么选？

- **轮询**：简单，适合少量、低频数据；缺点是阻塞/占 CPU。
- **中断**：每个字节或事件触发 ISR，适合中低速异步数据，但高吞吐时**中断**频繁。
- **DMA**：适合连续或大块收发，显著降低 CPU 搬运开销，常和空闲线/超时机配合处理不定长帧。

□□□ [项目高频] STM32 串口不定长接收怎么做？

常见方案是 UART RX DMA + IDLE 空闲线检测。DMA 负责连续收数据；检测到 IDLE 时读取 DMA 剩余计数，计算本次接收长度，再把数据交给解析任务。新版 HAL 也有 ReceiveToIdle 类接口。

□□□ [项目高频] UART 常见的 ORE、FE、NE、PE 分别是什么？

- **ORE (Overrun Error)**：上一个接收数据还没及时取走/处理，新数据又来了，可能造成数据丢失。
- **FE (Framing Error)**：停止位不符合预期，常见原因是波特率不匹配、线路异常或时序错误。
- **NE (Noise Error)**：采样期间检测到噪声。
- **PE (Parity Error)**：启用校验后，收到的数据校验不匹配。

面试注意：排查串口丢数据时，不要只看应用层缓冲区，也要统计这些硬件错误标志。

□□□ [常考] UART 的 RXNE/RXFNE 和 IDLE 有什么区别？

- **RXNE**（或带 FIFO 系列的 **RXFNE**）表示接收数据寄存器/FIFO 里有数据，适合逐字节**中断**接收。
- **IDLE** 表示接收线在收到数据之后出现了一段空闲，常用于判断一段不定长数据暂时结束。
- 工程上常把 DMA 连续接收与 IDLE 结合，减少每字节**中断**。

面试注意：IDLE 标志的清除方式、FIFO 行为和 HAL 接口在不同 STM32 系列上可能不同。

□□□ [项目高频] UART 的 TXE/TXFNF 和 TC 有什么区别？为什么 RS485 切方向要等 TC？

- **TXE/TXFNF** 表示发送数据寄存器或 FIFO 还能接收新数据，不代表最后一位已经从 TX 引脚发完。
- **TC (Transmission Complete)** 表示移位寄存器也完成，最后一个停止位已经发送结束。
- 半双工 RS485 拉低 DE 切回接收时，通常应等待 TC；如果只等 TXE，可能把最后一个字节尾部截断。

□□□ [常考] UART 和 USART 有什么区别？

UART 只描述异步收发器；USART 额外支持同步模式，可输出/输入时钟。STM32 的 USART 外设设在异步模式下就像普通 UART 使用。

□□□ [纠错必背] 同步通信和异步通信怎么区分？

在串行总线语境里，核心区别是是否有共享/显式时钟：SPI、I2C 有时钟线，UART 没有独立时钟线。它与“阻塞/非阻塞 API”是两个不同维度，不能混在一起。

1.5 UART / I2C / SPI 对比

接口	时钟	典型线数	双工	多设备	典型特点
UART	无独立时钟	TX/RX	全双工	通常点对点	简单，适合调试/模块通信
I2C	SCL	2	半双工式共享 SDA	支持多目标、多控制器	省线、有地址、ACK
SPI	SCLK	通常 4	全双工	一主多从常见	高速、协议简单、CS 占 IO

1.6 CAN

□□□ [高频] CAN 为什么适合汽车和工业现场？

- 差分物理层（经 CAN 收发器接 CAN_H/CAN_L），抗共模干扰能力强。
- 多节点共享总线，采用非破坏性仲裁，优先级确定。
- 协议自带 CRC、ACK、位监测、错误帧、错误计数等机制。
- 没有“某个固定主机”，任一节点在总线空闲时都可以发起发送。

□□□ [高频] MCU 的 CAN_TX/CAN_RX 能直接接 CAN_H/CAN_L 吗？

不能。MCU 内部通常只有 CAN 控制器，需要外接 CAN 收发器（如 TJA1050、SN65HVD 系列等）完成逻辑电平与差分总线之间的转换。

□□□ [常考] Classical CAN 有哪些帧？

主要包括数据帧、远程帧、错误帧、过载帧；帧与帧之间还有 Intermission 等间隔字段。面试不要把“间隔帧”当成与数据帧同类的 CAN 帧类型。

□□□ [纠错必背] 标准帧和扩展帧的区别？

Classical CAN 标准格式使用 11 位标识符，扩展格式使用 29 位标识符。两者数据场都只能是 0-8 字节。扩展帧只是 ID 更长，不代表数据能到 64 字节。

□□□ [纠错必背] CAN FD 为什么能到 64 字节？

CAN FD 是对 Classical CAN 的扩展，数据长度可到 64 字节，并可在数据阶段切换到更高位速率。它与“29 位扩展 ID”是两件不同的事情。

□□□ [高频] CAN 仲裁机制怎么工作？

节点边发边读。显性位 0 会覆盖隐性位 1。若节点发送隐性 1 却读到显性 0，就知道自己输掉仲裁并停止发送。一般来说标识符数值越小优先级越高；标准帧与扩展帧同基础 ID 竞争时还要看 SRR/IDE 位规则。

□□□ [常考] CAN 为什么叫“非破坏性仲裁”？

输掉仲裁的节点在仲裁字段就停止发送，而胜出的高优先级帧从头到尾没有被破坏，不需要碰撞后整帧重发。

□□□ [项目高频] CAN 总线为什么两端要 120Ω 终端电阻？

CAN 物理层常用约 120Ω 特性阻抗的双绞线，主干两端各接约 120Ω 终端，减少信号反射并提供合适的差分负载。不要在每个节点都并一个 120Ω。

□□□ [进阶] CAN 位时序一般怎么配？

由 CAN 内核时钟经预分频得到 Time Quantum，再由 Sync_Seg、BS1(TSEG1)、BS2(TSEG2) 组成一个 bit time。波特率 = CAN 时钟 / [Prescaler × (1 + BS1 + BS2)]。采样点通常落在一个 bit 的后段，具体按网络长度和收发器特性调整。

□□□ [常考] CAN 过滤器是干什么的？

硬件先按 ID/掩码筛选报文，只把关心的帧放入接收 FIFO/缓冲区，从而降低 CPU 软件过滤负担。列表模式适合列出明确 ID，掩码模式适合接收一组有共同位特征的 ID。

□□□ [进阶] CAN 常见错误机制有哪些？

包括位错误、填充错误、CRC 错误、格式错误、ACK 错误等。控制器维护发送/接收错误计数，根据错误程度进入 Error Active、Error Passive，严重时进入 Bus-Off；Bus-Off 后需要按控制器/协议要求恢复。

1.7 RS232 / RS485

□□□ [高频] RS232 和 UART 是什么关系？

UART 是数据帧/收发逻辑；RS232 是电气接口标准。MCU UART 要通过 MAX3232 等收发器才能变成 RS232 电平。

□□□ [高频] RS485 和 UART 是什么关系？

RS485 同样主要规定电气层。STM32 常用 UART + RS485 收发器组成链路，半双工时还需要一个 DE/RE 方向控制脚。上层帧格式可以是 Modbus RTU，也可以是自定义协议。

□□□ [常考] RS232 和 RS485 的主要区别？

- RS232：单端、常见点对点、全双工、距离相对短。
- RS485：差分、抗干扰强、可多节点、距离更远；两线制常用半双工，四线制也可全双工。
- RS485 的距离和速率互相制约，不应死背“固定 1.5 km”。

□□□ [项目高频] RS485 为什么要终端电阻和偏置电阻？

终端电阻用于匹配长线阻抗、减小反射；偏置 (fail-safe) 用于所有驱动器释放时让总线保持确定状态。具体阻值和是否需要外部偏置要看收发器是否自带 fail-safe 以及网络拓扑。

1.8 Modbus RTU / RS485

□□□ [高频] Modbus RTU 和 RS485 是一回事吗？

不是。Modbus 是通信协议/报文规则，RS485 主要是物理层电气标准。工程里常见“Modbus RTU over RS485”，所以两者经常一起出现。

- RS485 解决差分传输、电气驱动、多节点总线等问题。
- Modbus RTU 解决从站地址、功能码、数据字段、CRC、异常响应等问题。

□□□ [高频] Modbus RTU 一帧通常由哪些部分组成？

- 从站地址 Address：1 字节。
- 功能码 Function Code：1 字节。
- 数据 Data：长度随功能码变化。
- CRC16：2 字节，覆盖地址、功能码和数据；在线路上传输时低字节在前、高字节在后。

面试注意：不要把“寄存器地址”误认为固定占整帧第 3、4 字节；具体数据字段结构由功能码决定。

□□□ [高频] Modbus RTU 怎么判断一帧结束？

RTU 没有专门的固定帧尾字节，经典判帧依赖字符间隔。两帧之间需要足够长的总线静默时间，标准描述常用至少 3.5 个字符时间。

- 帧内部如果出现过长间隔，接收方可能把该帧判为无效/分帧。
- 在 MCU 上常用串口 IDLE、定时器超时或两者结合做接收判帧。

面试注意：不要把“3.5 字节”说成 3.5 个数据字节；它是按当前串口格式计算的字符时间。

□□□ [常考] Modbus RTU 的 CRC16 校验哪些内容？

从 Address 开始，一直到 Data 最后一个字节，CRC 自己的两个字节不参与计算。Modbus RTU 常用初值 0xFFFF，反射形式多项式常写作 0xA001。

- 发送时通常先发 CRC 低字节，再发高字节。
- 面试若让手写 CRC，不必死背每一步表法；要知道逐字节异或、按位右移和多项式异或的思路。

□□□ [高频] Modbus 最常见功能码有哪些？

- 0x01：读线圈 Coils。
- 0x02：读离散输入 Discrete Inputs。
- 0x03：读保持寄存器 Holding Registers。
- 0x04：读输入寄存器 Input Registers。
- 0x05：写单个线圈。
- 0x06：写单个保持寄存器。
- 0x0F：写多个线圈。
- 0x10：写多个保持寄存器。

□□□ [常考] Modbus 从站地址 0 有什么特殊含义？

串行 Modbus 中地址 0 常用于广播。主站可以向所有从站发送适合广播的写操作，从站执行后不应回复。

面试注意：广播如果大家都回复，就会造成总线冲突；因此“地址 0 读寄存器并等响应”不是正常用法。

□□□ [高频] Modbus 异常响应怎么识别？

从站不能正常执行请求时，会把原功能码的最高位置 1，也就是“原功能码 + 0x80”，然后在数据字段里返回异常码。

- 0x01: Illegal Function, 非法功能。
- 0x02: Illegal Data Address, 非法数据地址。
- 0x03: Illegal Data Value, 非法数据值。
- 0x04: Slave Device Failure, 从站设备故障。

□□□ [纠错必背] “40001 寄存器”在线路上是不是就发送地址 40001？

通常不是。4xxxx 是很多 Modbus 文档里的“参考编号/人类可读记法”，PDU 中真正发送的是协议地址字段，常见实现把 40001 对应到地址偏移 0。

- 不同厂家手册可能用 0-based 或 1-based 表示方式，最容易造成“地址差 1”。
- 实际调试要看设备手册明确写的是 Register Number、Reference 还是 Protocol Address。

面试注意：面试中答“40001 永远等于报文地址 0”也太绝对，最好补一句“以厂家地址约定为准”。

□□□ [项目高频] 两线 RS485 最容易踩的工程坑有哪些？

- 方向控制：发送前使能 DE，最后一个停止位真正发完后再切回接收，通常看 UART TC。
- 终端电阻：长线主干通常只在总线两端做匹配，不是每个节点都并 120Ω。
- 偏置电阻：用于所有驱动器释放时维持确定空闲状态；是否外加、阻值多大要结合收发器 fail-safe 和网络负载。
- 拓扑：优先总线/菊花链主干，支线尽量短；长距离高速场景不宜随意做大星形。

1.9 MQTT

□□□ [了解] MQTT 是什么？

MQTT 是面向消息的发布/订阅协议，常跑在 TCP 上。客户端连接 Broker，通过 Topic 发布和订阅消息，适合物联网和低带宽场景。

□□□ [常考] MQTT 的 QoS 有哪几级？

- QoS 0: 最多一次，尽力而为，不确认。
- QoS 1: 至少一次，有确认，可能重复。
- QoS 2: 恰好一次，握手更多，开销最大。

2. MCU 与 STM32

2.1 启动与内核

□□□ [高频] Cortex-M 上电后最先取什么？

复位后，处理器从**向量表**起始地址读取第 0 项作为初始 **MSP**（主栈指针），读取第 1 项作为 **Reset_Handler** 地址并跳转执行。向量表后续条目是异常和**中断**入口。

□□□ [高频] STM32 从 Reset_Handler 到 main() 一般做了什么？

- 初始化栈和 C 运行时环境。
- 把 .data 的初值从 **Flash** 拷到 **RAM**，清零 .bss。
- 执行 SystemInit/时钟相关初始化（具体顺序依启动文件和工具链）。
- 执行 C/C++ 运行库初始化，最后进入 main()。

面试注意：不要把“必然执行 __main”背成所有编译器通用结论；__main 是部分工具链的实现细节。

□□□ [常考] STM32 有哪几种启动方式？

经典 STM32F1 常见 Main **Flash**、System Memory、**SRAM** 三种启动映射，由 BOOT 引脚决定。较新的 STM32 可能通过 BOOT0 加 Option Bytes 等方式选择，不能把 F1 的 BOOT0/BOOT1 规则套到所有系列。

□□□ [纠错必背] System Memory Bootloader 是干什么的？

System Memory 中有 ST 出厂写好的 ROM **Bootloader**，可通过芯片支持的某些串行外设下载程序。不同 STM32 型号可能支持 **USART**、**USB DFU**、**CAN**、**I2C**、**SPI** 等不同组合，具体查 AN2606。

□□□ [高频] 什么是向量表重定位？

把**中断向量表**基地址改到新的位置。Cortex-M3/M4/M7 等通常通过 SCB->**VTOR** 设置。在 **Bootloader** + APP 场景中，APP 位于非 0x08000000 的地址时，需要让 **VTOR** 指向 APP 的向量表。

□□□ [进阶] 什么是 MSP 和 PSP？

MSP 是 Main Stack Pointer，异常处理默认使用 **MSP**；**PSP** 是 Process Stack Pointer，可供线程模式使用。RTOS 常让任务使用 **PSP**，内核/异常使用 **MSP**，从而把任务栈彼此隔离。

2.2 中断与 NVIC

□□□ [高频] 什么是中断？

外设或内核事件发生后，CPU 暂停当前线程代码，硬件自动保存部分现场并跳到对应 **ISR**；**ISR** 结束后通过异常返回恢复原上下文继续执行。

□□□ [项目高频] 中断里为什么不能做太多事情？

ISR 执行期间会增加其他**中断**/任务的响应延迟。原则是“快进快出”：读取状态、搬必要数据、清标志、发通知，把耗时解析、日志和算法放到主循环或 RTOS 任务里。

□□□ [高频] STM32 的抢占优先级和子优先级怎么理解？

优先级分组决定实现的优先级位如何分给 preemption priority 和 subpriority。抢占优先级更高的中断可以打断低抢占优先级 ISR；抢占级相同时，子优先级决定同时 pending 时谁先响应。数值越小通常优先级越高。

□□□ [高频] EXTI 和 NVIC 是什么关系？

- EXTI 负责检测外部/内部事件源，并产生中断请求或事件请求。
- NVIC 是 Cortex-M 内核里的中断控制器，负责中断使能、优先级、挂起状态和把异常送给 CPU。
- 可以理解为：EXTI 负责“产生请求”，NVIC 负责“CPU 什么时候响应这个请求”。

面试注意：GPIO 外部中断不是“只有 NVIC 配好就行”，通常还要配置 GPIO 输入、EXTI 映射/触发沿和 pending 清除。

□□□ [高频] 什么情况下高优先级中断才能打断当前 ISR？

必须同时满足：新中断已使能且处于 pending；它的抢占优先级高于当前 ISR；同时没有被 PRIMASK、BASEPRI 等屏蔽机制挡住。

- 子优先级主要用于相同抢占级、同时 pending 时决定先后，不负责产生嵌套抢占。
- STM32 中优先级数值通常越小越高。

□□□ [项目高频] 中断标志为什么一定要清？是不是统一“写 0 清除”？

如果外设的 pending/状态标志一直保留，CPU 退出 ISR 后可能马上再次进入中断，形成重复中断甚至“中断风暴”。

- 不同外设的清标志方式不同：可能是写 1 清除、读某寄存器后再读/写另一寄存器、由硬件条件自动清除等。
- 不能把“所有标志统一写 0 清”当通用规则，必须看具体系列 Reference Manual。
- 清除顺序还要避免丢掉在 ISR 处理期间新到来的事件。

□□□ [常考] NVIC 优先级数值为什么越小越高？

这是 Cortex-M NVIC 的编码约定。比如优先级 0 通常是最高可编程优先级。实际可用位数由芯片实现的 __NVIC_PRIO_BITS 决定。

□□□ [常考] SysTick 是什么？

Cortex-M 内核自带的 24 位递减计数器，可周期产生异常。裸机常用于 1ms 系统节拍；RTOS 常用它或其他定时器作为 Tick 源。

2.3 GPIO

□□□ [高频] GPIO 常见模式有哪些？

概念上包括输入、输出、复用功能、模拟；输入可配上拉/下拉/浮空，输出可选推挽/开漏。STM32F1 常被总结为“8 种 GPIO 模式”，但不同 STM32 系列寄存器模型不同，面试应先说概念再说具体型号。

□□□ [高频] 推挽输出和开漏输出区别？

- 推挽：内部上下管都能驱动，主动输出高和低，边沿快，适合普通数字输出。

- **开漏**：只能主动拉低，输出高需要外部/内部上拉，适合 I2C、线与、简单电平转换等。

□□□ [常考] 上拉输入、下拉输入、浮空输入怎么选？

上拉让悬空时默认读高，下拉让悬空时默认读低；浮空输入没有确定默认电平，未驱动时容易受干扰。按键等输入通常要有明确上拉/下拉。

□□□ [常考] 为什么 ADC 引脚通常设置为模拟模式？

模拟模式会关闭不必要的数字输入缓冲/输出驱动路径，减少数字电路对模拟信号的影响和额外功耗，并把引脚交给模拟外设。

2.4 时钟与定时器

□□□ [高频] 为什么学 STM32 一定要懂时钟树？

CPU、AHB/APB 总线和各外设都依赖时钟。UART 波特率、Timer 计数频率、ADC 时钟、USB 时钟等都由时钟树派生；时钟配错会导致“代码看着对但外设就是不对”。

□□□ [常考] PLL 是干什么的？

PLL 用参考时钟经倍频/分频得到目标系统时钟。配置时要满足输入范围、VCO 范围和各总线/Flash 等频率上限。不同 STM32 系列 PLL 结构差别很大。

□□□ [高频] 定时器基本计数频率怎么计算？

通用思路： $\text{TimerCounterClock} = \text{TimerClock} / (\text{PSC} + 1)$ ；更新事件频率约为 $\text{TimerCounterClock} / (\text{ARR} + 1)$ 。部分 STM32 在 APB 分频不为 1 时，定时器内核时钟可能是 APB 时钟的 2 倍，需要查参考手册。

□□□ [高频] PWM 的频率和占空比怎么理解？

PWM 通过定时器周期 ARR 和比较值 CCR 产生。边沿对齐时，频率主要由 TimerClock、PSC、ARR 决定；占空比约为 $\text{CCR} / (\text{ARR} + 1)$ ，具体要看 PWM 模式和计数方式。

□□□ [常考] 输入捕获有什么用？

在外部边沿到来时把当前计数器 CNT 自动锁存到 CCR，可测量脉冲宽度、周期、频率、占空比；比在 GPIO 中断里软件读时间戳更精确。

□□□ [常考] 输入捕获、输出比较、PWM 三者怎么区分？

- **输入捕获 Input Capture**：外部边沿到来时，把 CNT 自动锁存到 CCR，主要用于测频率、周期、脉宽。
- **输出比较 Output Compare**：CNT 与 CCR 匹配时触发事件，可翻转/置位/清零输出或产生中断。
- **PWM**：本质上是定时器比较功能的周期化应用，通过 ARR 定周期、CCR 定占空比。

□□□ [常考] 定时器“溢出”是什么意思？怎么利用它做更长时间计数？

向上计数时，CNT 计到自动重装值附近并发生更新事件后重新开始下一周期，通常会置更新标志 UIF。

- 可以在更新中断里维护一个软件高位计数器，把有限位宽的硬件计数扩展成更长时间范围。
- 计算时要注意 PSC、ARR、TimerClock，以及不同计数模式（向上/向下/中心对齐）带来的周期差异。

□□□ [了解] 编码器模式有什么用？

定时器可直接对正交编码器 A/B 相进行计数并判断方向，CPU 只需读取计数值，适合电机位置/速度检测。

2.5 ADC / DAC

□□□ [高频] ADC 分辨率是什么意思？

N 位 ADC 理论上把参考范围量化为 2^N 个码。比如 12 位、Vref=3.3V，理想 LSB 约为 3.3/4096 V。实际精度还受参考源、噪声、采样时间、非线性等影响。

□□□ [常考] ADC 采样时间为什么重要？

ADC 前端采样保持电容需要通过外部信号源阻抗充电。源阻抗越大，通常需要更长采样时间，否则采样电压来不及稳定，会产生误差。

□□□ [项目高频] ADC + DMA 的典型用法？

让 ADC 连续/扫描采样，DMA 自动把转换结果搬到数组；CPU 在半传输/全传输事件或任务里做滤波和计算，适合多通道周期采集。

□□□ [了解] DAC 是做什么的？

把数字码转换为模拟电压/电流。MCU 内置 DAC 可用于波形、基准、控制量输出；没有 DAC 的 MCU 也常用 PWM + RC 滤波近似模拟输出。

2.6 DMA

□□□ [高频] 什么是 DMA？

DMA (Direct Memory Access) 用于在外设与内存、内存与内存之间自动搬运数据。CPU 负责配置源/目的地址、方向、长度等，传输过程中不必逐字节参与，完成/半完成/错误时再通过中断处理。

□□□ [纠错必背] 用了 DMA，CPU 就完全不参与了吗？

不是。CPU 仍要初始化 DMA、启动传输、处理完成/错误事件，并保证缓存一致性（带 D-Cache 的 MCU 尤其重要）。DMA 减少的是“数据搬运”负担。

□□□ [常考] DMA 普通模式和循环模式区别？

普通模式搬完指定长度就停止；循环模式到达末尾后自动从头继续，常用于 ADC 连续采样、音频、UART 环形接收等。

□□□ [进阶] DMA 的半传输中断有什么用？

双半缓冲处理：DMA 填前半段时 CPU 处理后半段，填后半段时 CPU 处理前半段，形成流水线，适合连续采样和音频。

2.7 看门狗、Flash 与 Bootloader

□□□ [高频] 看门狗有什么用？

看门狗是独立/窗口式硬件计时器。正常系统周期性“喂狗”；如果软件死锁、跑飞导致未按时喂狗，硬件触发复位，提高系统自恢复能力。

□□□ [常考] IWDG 和 WWDG 思路有什么区别？

IWDG 通常由独立低速时钟驱动，鲁棒性高；WWDG 要求在规定时间窗口内刷新，既能发现“太晚没喂”，也能发现“喂得太早”，适合更严格的运行监控。具体系列实现看手册。

□□□ [常考] Flash 擦除和写入为什么不能像 RAM 一样？

Flash 通常要按页/扇区擦除，编程还受最小写入粒度、对齐、写入次数和电压条件限制。写 Flash 前要确认地址、擦除状态和执行期间对取指/中断的影响。

□□□ [高频] Bootloader + APP 跳转的关键步骤有哪些？

- 确认 APP 地址和向量表合法，通常先检查初始栈指针是否落在 SRAM 范围。
- 停止 Bootloader 中不再需要的外设/DMA/定时器，必要时关闭或清 pending 中断。
- 设置向量表到 APP 地址（或保证 APP 启动后立即设置）。
- 从 APP 向量表读取初始 MSP 和 Reset_Handler，设置 MSP 后跳转到 Reset_Handler。

□□□ [高频] 为什么不建议直接“调用 APP 的 main()”？

main() 不是应用的真正复位入口。APP 需要自己的启动流程、栈、.data/.bss 和 SystemInit 等初始化，因此应跳到 APP 的 Reset_Handler，而不是普通函数调用 main。

2.8 HAL、LL、寄存器与调试

□□□ [常考] HAL、LL、寄存器开发有什么区别？

- **HAL**：抽象高、可读性和移植性好，上手快，但代码体量和开销相对大。
- **LL**：更接近寄存器，封装薄，适合性能敏感或需要精细控制。
- **寄存器**：控制最直接，但开发维护成本高、跨系列迁移差。
- **工程里可以混用**：大部分 HAL，关键路径 LL/寄存器。

□□□ [项目高频] STM32 程序“跑飞”怎么排查？

- 先确认复位原因寄存器、HardFault、看门狗、供电和时钟。
- 用调试器看 PC/LR/SP、Fault Status 寄存器和调用栈。
- 检查数组越界、野指针、栈溢出、错误函数指针、并发问题。
- 必要时增加断言、日志、栈水位、硬 Fault 捕获信息。

□□□ [常考] 芯片选型主要看什么？

内核性能、Flash/RAM、GPIO 数量、定时器/ADC/通信外设、封装、工作温度、电源、电气特性、成本、供货周期、生态和认证要求。先根据项目资源预算选系列，再根据封装/外设数量落到具体料号。

3. C 语言高频

3.1 编译、链接与存储

□□□ [高频] 一个 .c 文件到可执行文件经历什么？

预处理 -> 编译 -> 汇编 -> 链接。预处理处理宏、条件编译和头文件；编译生成汇编/中间代码；汇编生成目标文件；链接解析符号和重定位，合并目标文件/库形成最终镜像。

□□□ [高频] 程序常见的段有哪些？

- **.text**: 代码和常量的一部分，通常位于 **Flash**/只读区。
- **.rodata**: 只读常量（是否独立取决于工具链）。
- **.data**: 有非零初值的全局/静态变量，运行时在 **RAM**，初值存于 **Flash**。
- **.bss**: 零初始化或未显式初始化的全局/静态变量，启动时清零。
- **heap**: 动态分配区域；**stack**: 函数调用、局部自动变量、保存现场等。

□□□ [常考] map 文件是干什么的？面试时会看哪些信息？

map 文件是链接器生成的镜像布局报告，可以看到符号最终地址、各段大小、每个目标文件/库占用了多少 **Flash** 和 **RAM**。

- 定位“**Flash**/**RAM** 为什么突然变大”。
- 确认函数或变量到底链接到了哪里。
- 结合 **HardFault** 的 **PC/LR** 地址反查符号。
- 检查某个库/模块是否被真正链接进最终镜像。

□□□ [常考] 链接脚本（Linker Script / Scatter File）主要控制什么？

- 描述 MCU 的 **Flash**、**RAM** 等内存区域及其起始地址和长度。
- 决定 **.text**、**.rodata**、**.data**、**.bss**、栈、堆等段放到哪里。
- **Bootloader** + APP、外部 **Flash**、**RAM** 特殊区域、固定地址参数区等场景都可能需要修改链接布局。

面试注意: GCC 常见 **.ld**；Keil/Arm 工具链常见 **scatter file**，语法不同但核心目的相同。

□□□ [项目高频] 为什么开启 -O2/-O3 后调试时变量“消失”或单步顺序很奇怪？

优化器可能把变量放进寄存器、合并表达式、删除无用代码、内联函数、重排指令，所以源码行和最终机器指令不再一一对应。

- 调试版通常降低优化等级以便观察。
- **volatile** 只解决特定“可观察访问”问题，不是让所有变量都方便调试的开关。
- 遇到“**Debug** 正常、**Release** 异常”还要排查未初始化变量、越界、竞态、未定义行为和时序依赖。

□□□ [高频] 全局变量、局部变量、静态变量主要区别？

要分“作用域、链接属性、存储期”回答。普通局部变量通常块作用域、自动存储期；文件作用域全局变量具有静态存储期；**static** 局部变量仍是块作用域，但具有静态存储期；文件作用域 **static** 会获得内部链接，只在本翻译单元可见。

□□□ [高频] static 关键字有哪些作用？

- 函数内部 static 局部变量：作用域仍在块内，但生命周期贯穿整个程序，只初始化一次。
- 文件作用域 static 变量：内部链接，限制在当前 .c 文件。
- static 函数：内部链接，只能在当前翻译单元引用。

□□□ [常考] extern 是什么？

extern 声明一个具有外部链接的对象或函数“定义在别处”，让当前文件可以引用。声明不等于分配一个新的全局对象定义。

□□□ [常考] const 在 C 里是什么意思？

const 表示通过该左值不能修改对象，不等价于“编译期常量”，也不保证一定放在 Flash。对指针参数加 const 常用于表达只读接口。

□□□ [高频] volatile 到底保证什么？

volatile 告诉编译器：对该对象的访问是可观察的，不能把这些访问随意删掉或合并成普通缓存值。典型用途是内存映射外设寄存器，以及主程序与 ISR 共享且由异步事件修改的简单标志。

□□□ [纠错必背] volatile 不保证什么？

- 不保证复合操作原子性，例如 count++ 仍可能被打断。
- 不提供互斥，也不是线程同步原语。
- 不能笼统说“每次一定访问主存”；在带缓存/乱序的系统上，内存可见性和顺序还可能需原子操作、屏障或 OS 同步机制。

3.2 指针与数组

□□□ [高频] 数组和指针是什么关系？

数组是一块连续对象；指针是存地址的变量。数组表达式在大多数表达式中会转换为指向首元素的指针，但数组本身不是指针。sizeof(array) 和 &array 都能体现区别。

□□□ [纠错必背] 为什么说“数组名就是指针常量”不严谨？

因为数组名没有“指针对象”本身的存储，也不能给它赋新地址；在 sizeof、取地址 &、字符串初始化等场景不会发生数组到指针的转换。面试最好说“多数表达式中数组会衰减为首元素指针”。

□□□ [高频] 函数参数写成 int a[]，在函数里 sizeof(a) 能得到数组长度吗？

不能。数组作为函数参数时，参数声明会调整为指针类型，本质上等价于 int *a，因此 sizeof(a) 得到的是指针大小，不是调用者数组的总字节数。

- 数组长度通常要额外作为参数传入。
- 只有在数组对象本身仍处于当前作用域、尚未发生数组到指针转换时，sizeof(array) 才能得到整个数组大小。

面试注意：“形参写了 [] 看起来像数组”不代表函数真正收到了一整个数组对象。

□□□ [高频] `char *p = "abc"` 和 `char a[] = "abc"` 有什么区别?

- `char *p` 指向字符串字面量。C 中通过 `p` 去修改字面量内容属于未定义行为，工程上应把这类指针写成 `const char *p`。
- `char a[]` 会创建一个可写字符数组，并把 `a`、`b`、`c`、`\0` 拷入数组，因此 `a[0] = 'A'` 是合法的。
- `sizeof(p)` 是指针大小；`sizeof(a)` 是数组总大小，这里通常为 4。

□□□ [常考] 二级指针 `int **pp` 什么时候会用到?

二级指针本质上是“指向指针的指针”。最典型的用途是让函数修改调用者手里的指针本身，而不仅仅是修改指针指向的数据。

- 动态申请内存时，把指针地址传给初始化函数。
- 链表头结点可能发生变化时，传 `Node **head`。
- 指针数组、二维动态结构、驱动对象句柄等场景也会见到。

```
int alloc_buf(uint8_t **buf, size_t n);
```

□□□ [常考] 指针 `p + 1` 到底移动几个字节?

取决于 `p` 指向的类型。若 `p` 是 `T*`，则 `p + 1` 前进一个 `T` 对象的距离，也就是通常前进 `sizeof(T)` 个字节。

- `char*` 加 1 通常前进 1 字节；`uint32_t*` 加 1 通常前进 4 字节。
- 标准 C 的指针运算应限制在同一个数组对象及其“尾后一个位置”范围内；尾后指针可以形成和比较，但不能解引用。
- `void*` 上直接做 +1 不是 ISO C 的通用写法，部分编译器把它当扩展。

□□□ [常考] 指针的大小由什么决定?

由目标 ABI/地址宽度决定，不由指向类型决定。同一数据模型下 `int*`、`char*` 等对象指针通常大小相同，但标准层面不要把所有平台都绝对化。STM32 32 位地址空间中对象指针通常为 4 字节。

□□□ [高频] 什么是野指针和悬空指针?

野指针常指未初始化或指向无效区域的指针；悬空指针是原对象生命周期结束后仍保留其旧地址的指针，例如 `free` 后继续使用、返回局部变量地址。解决方法是初始化、严格管理生命周期、释放后按场景置 `NULL`，并做边界检查。

□□□ [高频] 常量指针和指针常量怎么区分?

`const int *p;` // 指向 `const int`：不能通过 `p` 改 `*p`，`p` 可改

`int *const p2 = &x;` // `const` 指针：`p2` 不能改指向，`*p2` 可改

`const int *const p3 = &x;` // 两者都限制

□□□ [常考] 数组指针和指针数组怎么区分?

`int (*p)[10];` // `p` 是指针，指向“10 个 `int` 的数组”

`int *p2[10];` // `p2` 是数组，里面有 10 个 `int*`

□□□ [高频] 函数指针和指针函数区别？

函数指针是“指向函数的指针”，常用于回调、驱动抽象、状态表；指针函数是“返回值为指针的函数”。看括号最直接。`int *func(int x);` // 指针函数：返回 `int*`
`int (*funcp)(int x);` // 函数指针：指向返回 `int` 的函数

□□□ [常考] 回调函数是什么？

把函数地址作为参数或成员传给另一模块，在事件发生时由对方调用。典型场景：**HAL 中断回调**、驱动层事件通知、状态机、协议栈。

3.3 宏、头文件与类型

□□□ [常考] `#include <...>` 和 `#include "..."` 有什么区别？

搜索路径规则由实现决定；常见编译器中，双引号形式会先从当前文件/工程指定路径查找，再回退系统路径；尖括号主要走系统/配置的 `include` 路径。区别不是“哪个搜索更快”。

□□□ [高频] 如何防止头文件重复包含？

标准做法是 `include guard`。很多编译器也支持 `#pragma once`，但它不是 ISO C 标准要求。`#ifndef MODULE_H`
`#define MODULE_H`
`// declarations`
`#endif`

□□□ [高频] `#define` 和 `typedef` 的区别？

- `#define` 是预处理文本替换，没有 C 类型语义。
- `typedef` 在语言语法层面创建类型别名，遵循作用域规则。
- 对复杂指针/函数类型，`typedef` 通常更安全清晰；宏可能因替换和优先级产生陷阱。

□□□ [常考] `#define` 和 `const` 怎么选？

数值常量若需要类型检查和调试符号，优先 `const/enum`；宏适合条件编译、编译期拼接和真正的宏函数。注意 C 中文件作用域 `const` 默认仍可能具有外部链接（与 C++ 不同），具体设计要配合 `static/extern`。

□□□ [常考] 宏函数为什么要给参数和整体加括号？

宏是文本替换，不加括号容易受运算符优先级影响。但即使加括号，也要警惕参数被求值多次，例如 `MIN(i++, j++)`。`#define MIN(a,b) (((a) < (b)) ? (a) : (b))`

□□□ [常考] `inline` 一定会内联吗？

不一定。`inline` 是优化和链接语义相关的关键字，编译器可以选择不展开；开启优化时普通函数也可能被内联。面试中不要把 `inline` 说成“保证消除函数调用”。

3.4 内存与字符串

□□□ [高频] 堆和栈有什么区别？

- **栈**：通常由编译器/CPU 按调用约定自动使用，用于调用帧、局部自动对象、保存现场；空间通常有限。
- **堆**：由动态内存分配器管理，C 中 **malloc/calloc/realloc** 申请，**free** 释放。
- 裸机 MCU 也有栈，并不需要操作系统；任务栈在 RTOS 中由内核为每个任务分配/管理。

□□□ [纠错必背] C 语言有哪些动态内存函数？

malloc、**calloc**、**realloc**、**free**。**new/delete** 属于 C++，不要写进 C 语言答案。实时 MCU 项目是否允许频繁动态分配，要看确定性、碎片和规范要求。

□□□ [常考] malloc、calloc、realloc 怎么区分？realloc 最容易踩什么坑？

- **malloc(n)**：申请 n 字节，内容未初始化。
- **calloc(count, size)**：申请 $\text{count} \times \text{size}$ 字节，并把返回区域按字节清零。
- **realloc(ptr, new_size)**：调整已有分配块大小，可能原地扩展，也可能搬到新地址。
- **realloc** 失败时通常返回 NULL，而旧 ptr 仍然有效；不要直接写 **ptr = realloc(ptr, n)**，否则失败时可能丢失旧指针。

```
void *tmp = realloc(ptr, new_size);  
if (tmp != NULL) ptr = tmp;
```

面试注意：实时 MCU 项目里是否使用动态内存，仍要考虑碎片、确定性和长期运行风险。

□□□ [高频] 什么是内存泄漏？

申请到的动态内存不再可达或不再需要，却没有释放，导致可用堆逐渐减少。长期运行的嵌入式系统尤其危险。

□□□ [常考] 什么是内存碎片？

堆中虽然总空闲空间不少，但被分成很多小块，导致无法满足较大的连续分配。频繁申请/释放不同大小块容易产生外部碎片。

□□□ [高频] sizeof 和 strlen 有什么区别？

sizeof 是运算符，返回对象/类型占用字节数；多数情况下编译期可确定，但 VLA 等情况可在运行期求值。**strlen** 是库函数，从首字符一直数到 `\0`，不包含终止符。

□□□ [常考] 为什么 sizeof("abc") 是 4，而 strlen("abc") 是 3？

字符串字面量包含结尾的 `\0`，因此数组大小是 4；**strlen** 只统计 `\0` 之前的字符。注意如果把字符串传给函数后参数已经变成 `char*`，**sizeof** 参数得到的是指针大小。

□□□ [高频] memcpy、memmove、strcpy 有什么区别？

- **memcpy**：按字节复制指定长度，不关心 `\0`；源和目标重叠时行为未定义。
- **memmove**：同样按字节复制，但允许源和目标区域重叠。
- **strcpy**：复制 C 字符串直到包含结尾 `\0`；目标缓冲区不足会越界，应谨慎使用。

□□□ [纠错必背] `memset(arr, 1, sizeof(arr))` 能把 `int` 数组每个元素都设成 1 吗?

不能。`memset` 是“按字节”填充值。若 `int` 为 32 位，填 `0x01` 后常见结果是每个元素变成 `0x01010101`，而不是整数 1。

- 给普通整数缓冲区清零时，`memset(..., 0, ...)` 很常见。
- 设置任意非零整数值应循环赋值或使用合适的初始化逻辑。

□□□ [常考] `signed` 和 `unsigned` 混合比较为什么容易出错?

因为 C 的通常算术转换可能把有符号数转换为无符号数。负数一旦转成无符号类型，可能变成很大的正数，比较结果就会与直觉相反；长度、索引常用 `size_t`，与 `int` 混用时尤其要注意。

3.5 结构体、联合体与对齐

□□□ [高频] 什么是内存对齐?

编译器按目标 ABI 的对齐要求放置对象和结构体成员，以便 CPU 高效/合法访问。成员之间和结构体尾部可能插入 `padding`。

□□□ [纠错必背] `struct { char a; double b; }` 为什么常见大小是 16?

不是“`char` 占了 8 字节”，而是 `a` 仍占 1 字节，后面插入若干 `padding`，让 `b` 从满足 8 字节对齐的地址开始；结构体总大小还要向最大对齐要求取整。具体结果依 ABI。

□□□ [高频] 结构体和联合体区别?

`struct` 每个成员有各自存储空间，大小包含成员和 `padding`；`union` 的成员共享同一段存储，大小至少能容纳最大成员，并满足最大对齐要求，因此可能比“最大成员裸大小”更大。

□□□ [项目高频] 为什么嵌入式通信结构体不能直接随便 `memcpy` 上线?

因为可能存在 `padding`、大小端、编译器 ABI、字段宽度和对齐差异。网络/总线协议应显式定义字节序和字段长度，逐字段序列化，或使用明确 `packing` 规则并评估未对齐访问风险。

3.6 数据结构与工程写法

□□□ [常考] 数组和链表的区别?

数组连续、随机访问 $O(1)$ 、缓存友好，但中间插入删除需要搬移；链表节点可分散，已知节点位置时插入删除可 $O(1)$ ，但定位通常 $O(n)$ 、额外占指针空间、缓存局部性差。不能简单说“链表增删改查都比数组快”。

□□□ [高频] 数据结构里的“栈”和“队列”有什么区别?

- 栈 `Stack`：后进先出 LIFO，典型操作 `push/pop`。
- 队列 `Queue`：先进先出 FIFO，典型操作 `enqueue/dequeue`。
- 嵌入式里“CPU/任务栈”和“数据结构栈”概念相关但不是同一道题；`FreeRTOS Queue` 也不只是一个普通数组，而是带同步/阻塞语义的内核对象。

□□□ [常考] 单链表插入删除真的是 $O(1)$ 吗?

要看前提。如果已经拿到了目标节点的前驱位置，插入/删除链接本身可以 $O(1)$ ；如果先要从头查找目标位置，查找通常是 $O(n)$ 。

- 链表随机访问不是 $O(1)$ 。
- MCU 中链表还要考虑节点动态分配、碎片、指针开销和缓存局部性。

□□□ [常考] 常见复杂度 $O(1)$ 、 $O(n)$ 、 $O(\log n)$ 怎么理解？

$O(1)$ 表示工作量基本不随数据规模增长，如数组下标访问； $O(n)$ 表示大体线性增长，如无序数组查找； $O(\log n)$ 表示每一步按比例缩小问题，如有序数组二分。嵌入式还要同时看最坏执行时间、RAM 和实时性。

□□□ [高频] 什么是环形缓冲区？

用固定数组配合读写索引把末尾和开头逻辑连接起来，适合 UART、DMA、音频、日志等流数据。关键是定义空/满判定和并发策略。常见做法预留一个单元区分满与空，或额外维护 count。

□□□ [项目高频] 环形缓冲区为什么适合串口？

中断/DMA 负责快速把新字节写入环形区，主循环/任务异步解析；生产者和消费者解耦，可以吸收瞬时突发流量，减少丢包。

□□□ [常考] 什么是状态机？

把系统行为拆成有限状态（State）和事件（Event），在明确条件下进行状态转移。相比大量 if/else，状态机更容易描述协议流程、按键/UI、电机控制和设备工作模式。

□□□ [高频] 大小端是什么？

多字节对象在内存中的字节排列：小端是低有效字节放低地址，大端是高有效字节放低地址。STM32 Cortex-M 常用小端。网络协议经常规定网络字节序为大端，需要显式转换。

□□□ [高频] 位操作为什么在嵌入式里很重要？

寄存器和协议字段经常按位表示状态。常用操作：置位 $x |= \text{mask}$ ；清位 $x \&= \sim \text{mask}$ ；翻转 $x \wedge= \text{mask}$ ；测试 $(x \& \text{mask}) != 0$ 。修改寄存器时要注意读改写、副作用和并发。

□□□ [高频] 什么叫竞态条件？

程序结果依赖多个执行上下文的不可控执行顺序。例如任务和 ISR 同时修改共享变量。解决要根据场景使用临界区、原子操作、互斥量、队列/通知或短暂关中断，而不是只加 volatile。

4. FreeRTOS

4.1 任务与调度

□□□ [高频] FreeRTOS 任务有哪些主要状态?

Running（运行）、Ready（就绪）、Blocked（阻塞）、Suspended（挂起）。任务等待延时、队列、信号量等通常进入 Blocked；vTaskSuspend() 进入 Suspended。

□□□ [高频] FreeRTOS 抢占式调度怎么理解?

当更高优先级任务变为 Ready，若启用抢占，调度器会让它尽快抢占当前低优先级任务。高优先级任务如果一直不阻塞/不延时，会饿死低优先级任务。

□□□ [纠错必背] 同优先级任务怎么轮转?

启用 configUSE_TIME_SLICING 且启用抢占时，多个同优先级 Ready 任务可在 Tick 边界轮转。一个时间片通常就是一个 Tick；Tick 是 1ms 还是 10ms 由 configTICK_RATE_HZ 决定，不是固定 1ms。

□□□ [纠错必背] vTaskDelay() 是“协作式调度”吗?

不是。vTaskDelay() 的本质是让当前任务进入 Blocked 一段时间，从 Ready 集合移出。它在抢占式系统里也大量使用。协作式/抢占式是调度器策略，不应和“主动延时”混为一谈。

□□□ [项目高频] vTaskDelay() 和 vTaskDelayUntil() 区别?

vTaskDelay() 以“调用时刻”为基准延时，任务执行时间会累积到周期误差；vTaskDelayUntil() 以固定参考节拍形成绝对周期，更适合周期采样/控制任务。

□□□ [高频] 什么时候会发生任务切换?

可能来自 Tick 时间片、当前任务阻塞/延时、主动 yield、更高优先级任务被任务或 ISR 唤醒、任务优先级变化等。Cortex-M 端口通常最终通过 PendSV 完成上下文切换。

4.2 SysTick / PendSV / SVC

□□□ [高频] FreeRTOS 为什么用 PendSV 做任务切换?

PendSV 是 Cortex-M 可挂起异常，RTOS 通常把它配置为最低优先级。这样高优先级硬件中断先处理，等异常嵌套退出后再做耗时的上下文切换，减少对实时中断的干扰。

□□□ [高频] SysTick 和 PendSV 分别做什么?

SysTick（或其他 Tick 源）提供周期节拍，更新延时/超时并可能请求调度；PendSV 执行真正的任务上下文保存与恢复。任务切换不只由 SysTick 发起。

□□□ [常考] SVC 在 FreeRTOS Cortex-M 端口里有什么作用?

SVC（Supervisor Call）常用于启动第一个任务/进入调度器等特权切换流程。具体端口实现不同，但 SVC、PendSV 通常是 RTOS 的核心异常机制，不是“移植时把它屏蔽掉”。

□□□ [进阶] 上下文切换保存哪些东西？

Cortex-M 异常入栈会自动保存 R0-R3、R12、LR、PC、xPSR 等基本现场；RTOS 端口再软件保存 R4-R11 等需要保持的寄存器，并保存任务栈指针到 TCB。带 FPU 时还要考虑浮点上下文。

4.3 队列、信号量、互斥量、事件组、通知

□□□ [高频] FreeRTOS 队列是干什么的？

队列用于任务/ISR 与任务之间传递定长消息，并自带阻塞等待和唤醒机制。发送时通常把数据复制进队列，不只是传一个“通知”。

□□□ [高频] 二值信号量的典型用途？

同步事件。例如 ISR 采集完成后 give，一个处理任务 take 后被唤醒。它通常不用于保护“必须归还给持有者”的资源。

□□□ [高频] 互斥量和二值信号量区别？

互斥量用于互斥访问共享资源，并带**优先级继承**机制；二值信号量更适合事件同步。互斥量应该由获取它的任务释放，ISR 不能用普通 mutex API。

□□□ [高频] 什么是优先级反转？

低优先级任务持有锁，高优先级任务等待该锁；此时中优先级任务不断运行，使低优先级任务没机会释放锁，导致高优先级任务被间接阻塞。互斥量的**优先级继承**会临时提高持锁任务优先级，缩短反转时间。

□□□ [常考] 计数信号量有什么用？

可表示多个相同资源的可用数量，或者累计多次事件。例如固定数量的缓冲块、生产者产生了 N 个待处理事件。

□□□ [常考] 事件组是什么？

用一组 bit 表示多个事件标志。任务可以等待“任意一个 bit”或“所有指定 bit”满足，适合多个条件组合；它不承载大块数据。

□□□ [常考] 任务通知有什么优势？

通知值直接存放在任务 TCB 中，不需要单独创建队列对象，通常比信号量/队列更轻量更快。适合一对一事件、计数、位标志或少量值传递。

机制	主要用途	传数据	关键特点
Queue	消息传递	是	复制数据，可阻塞等待
Binary Semaphore	事件同步	通常否	适合 ISR->Task / Task->Task 同步
Mutex	互斥资源	否	优先级继承，任务持有/释放
Event Group	多条件同步	bit 标志	等任意/全部事件
Task Notification	轻量一对一通知	值/计数/bit	对象少，性能高

4.4 ISR 与临界区

□□□ [高频] 为什么 ISR 里要用 xxxFromISR API?

ISR 上下文不能像普通任务那样阻塞，FreeRTOS 提供专用 FromISR API，并通过一个参数告诉你是否唤醒了更高优先级任务；必要时在退出 ISR 前请求一次调度。

□□□ [常考] portYIELD_FROM_ISR() 是干什么的?

如果 ISR 通过队列/信号量/通知唤醒了比当前任务优先级更高的任务，就请求在**中断**退出后尽快切换到该高优先级任务。Cortex-M 端口通常通过挂起 PendSV 实现。

□□□ [项目高频] FreeRTOS 中断优先级为什么容易踩坑?

调用 FreeRTOS FromISR API 的**中断**，其优先级必须处在内核允许的范围（受 configMAX_SYSCALL_INTERRUPT_PRIORITY 或库层宏限制）。数值越小优先级越高，因此“太高优先级”的 ISR 反而不能调用这些内核 API。

□□□ [高频] 临界区和互斥量怎么选?

临界区适合非常短、必须原子执行且常与**中断**并发有关的代码；互斥量适合任务之间保护可能持续较久的共享资源。长时间关**中断**会严重影响实时性。

4.5 内存与任务栈

□□□ [高频] FreeRTOS heap_1 到 heap_5 怎么记?

- heap_1: 只分配不释放，最简单、确定性好。
- heap_2: 可释放，但不合并相邻空闲块，容易碎片；适合反复分配相同/少数固定尺寸。
- heap_3: 封装标准库 malloc/free，需要标准库线程安全处理。
- heap_4: 可分配释放，并合并相邻空闲块，常用。
- heap_5: 在 heap_4 思路支持多个不连续内存区域。

□□□ [项目高频] 任务栈太小会怎样?

局部变量、函数嵌套、**中断**/库调用等可能把任务栈压穿，造成随机崩溃或破坏其他内存。可开启栈溢出检测，并用 uxTaskGetStackHighWaterMark 类接口观察历史最小剩余栈。

□□□ [常考] 为什么不建议任务里放特别大的局部数组?

局部自动数组直接占任务栈。大缓冲区更适合放静态区、专用内存池或经过设计的共享缓冲区，并明确并发保护和所有权。

□□□ [常考] FreeRTOS 空闲任务有什么作用?

系统自动创建最低优先级 Idle Task，保证永远有可运行任务；它还负责回收已删除任务的动态资源。可以配置 Idle Hook 做轻量后台/低功耗入口，但不能长时间阻塞。

□□□ [常考] 软件定时器由谁执行回调?

FreeRTOS 软件定时器回调由 Timer Service/Daemon Task 执行，不是在硬件**中断**里。回调应尽量短，不能长时间阻塞，否则会影响其他软件定时器处理。

□ □ □ [纠错必背] FreeRTOS 和 Linux 的实时性怎么比较？

FreeRTOS 是面向 MCU 的实时内核，调度路径短、可配置、易做确定性分析；Linux 功能和资源管理更复杂，普通内核不保证硬实时，PREEMPT_RT 可显著改善实时性。是否达到硬实时要看整个系统最坏时延，而不是只看 OS 名称。

5. Linux 基础（MCU 岗够用版）

□□□ [常考] 进程和线程的区别？

进程拥有独立的虚拟地址空间和系统资源上下文；同一进程的线程共享地址空间、文件描述符等大部分进程资源，但每个线程有自己的栈和寄存器上下文。线程切换通常比进程切换轻，但两者都由内核调度。

□□□ [了解] fork() 和 exec() 分别做什么？

fork() 创建一个新进程，子进程最初拥有父进程地址空间的逻辑副本（现代系统通常用写时复制）；exec() 用新程序映像替换当前进程的用户态地址空间。Shell 常用 fork + exec 启动程序。

□□□ [了解] Linux 进程常见状态怎么理解？

面试常见可说：运行/可运行、可中断睡眠、不可中断睡眠、停止/跟踪、僵尸等。不要死背一个跨所有 OS 都完全相同的“五状态模型”。

□□□ [常考] 进程间通信 IPC 有哪些？

管道/FIFO、消息队列、共享内存、信号量、信号、socket 等。共享内存吞吐高，但需要额外同步；socket 跨机器也能用。

□□□ [常考] 用户态和内核态有什么区别？

用户态权限受限，不能直接访问内核资源；系统调用让进程受控地进入内核完成文件、网络、内存等操作。嵌入式 Linux 驱动主要运行在内核态。

□□□ [了解] mmap() 是什么？

把文件、设备或匿名内存映射到进程虚拟地址空间，之后可通过普通内存访问读写映射区域。常用于文件映射、共享内存和某些设备驱动接口。

6. 项目追问：把“会背”变成“会做”

□□□ [必练] 面试官问“你项目里的串口怎么做的？”怎么答？

建议按链路回答：用途 -> 波特率/帧格式 -> 接收方式 -> 缓冲 -> 帧解析 -> 异常处理。比如：“USART2 115200 8N1，RX 用 DMA + IDLE 做不定长接收，数据先进入环形缓冲区，任务按帧头/长度/CRC 解析，溢出和校验错误会丢帧并计数。”

□□□ [项目高频] 怎么防止串口丢数据？

- 减少 ISR 工作量，优先使用 DMA/环形缓冲。
- 生产者和消费者解耦，保证缓冲容量覆盖最坏突发量。
- 协议层设计长度、超时、CRC、序号或重同步机制。
- 监测 ORE/FE/NE 等错误并记录统计，定位是软件处理不及时还是物理链路问题。

□□□ [项目高频] I2C 传感器偶尔读失败怎么排查？

- 确认地址是 7 位还是“已经左移一位的 8 位地址”。
- 检查上拉电阻、SCL/SDA 高电平、电源和线长。
- 用逻辑分析仪看 START、地址、ACK、寄存器地址和 NACK 发生位置。
- 处理总线忙/死锁，必要时 GPIO 恢复时钟后重新初始化外设。
- 核对设备上电等待时间、重复起始条件和寄存器读写时序。

□□□ [项目高频] SPI 屏幕/Flash 读写不对怎么排查？

优先核对 SPI Mode、最大频率、CS 时序、bit order、命令/地址长度、dummy cycle，再看逻辑分析仪波形。不要先怀疑 HAL 库。

□□□ [项目高频] 为什么用 DMA 还可能丢串口数据？

DMA 只解决搬运，不保证你的上层处理及时。可能因为 DMA 缓冲太小、重启接收存在空窗、缓存一致性、数据覆盖、IDLE 处理错误、任务消费太慢或硬件 ORE。设计要看完整数据通路。

□□□ [项目高频] 任务之间共享一个大缓冲区怎么设计？

先确定所有权。最稳妥是单一生产者/消费者用队列传“缓冲区指针+长度”，或用内存池。若确实多任务同时访问同一对象，再用 mutex/临界区保护，并避免持锁期间做慢 IO。

□□□ [项目高频] 看门狗应该谁来喂？

不建议某一个永远循环的低价值任务无条件喂狗。更可靠的做法是由监控任务确认关键任务都在期限内“报活”后再统一喂狗，这样关键任务卡死才能真正触发复位。

□□□ [项目高频] HardFault 怎么定位？

捕获异常栈中的 PC/LR，读取 CFSR/HFSR/BFAR/MMFAR 等故障寄存器，结合 map/elf 反查代码位置；重点排查非法地址、未对齐/总线错误、栈溢出、错误函数指针和损坏返回地址。

□□□ [高频] 为什么项目里需要超时机制？

外设、通信和锁都可能异常。如果无限等待 ACK、互斥量或设备 Ready，整个任务可能永久卡死。工程接口通常要定义 timeout、错误码、重试次数和故障降级策略。

□□□ [必练] “我会 STM32F103”，怎么说得更像工程能力？

把芯片名换成能力描述：熟悉 Cortex-M MCU 开发流程，能使用 GPIO/Timer/ADC/UART/I2C/SPI/CAN/DMA/中断，能用 CubeMX/HAL 快速配置并根据 Datasheet/Reference Manual 定位问题；换型号主要重新核对时钟、引脚复用、DMA 映射和外设差异。

□□□ [必练] 一个新传感器你没用过，怎么快速驱动？

- 先看供电、电平、引脚和通信接口。
- **Datasheet** 找设备地址、寄存器表、上电时序和初始化序列。
- 先完成最小读 ID/WHO_AM_I。
- 再封装 read/write register，最后实现功能配置和数据换算。
- 用逻辑分析仪验证底层通信，避免直接在业务代码里猜。

□□□ [必练] 项目里为什么用 FreeRTOS，而不是 while(1)？

不能回答“因为项目复杂”。要说具体收益：多个不同周期/优先级功能需要独立调度；通信接收需要阻塞等待事件；任务间用队列/通知解耦；高优先级控制任务需要可预测响应。若项目很简单，裸机反而可能更合适。

□□□ [必练] 面试官问“遇到过什么 Bug？”怎么组织答案？

用 STAR 的工程版本：现象 -> 如何复现/测量 -> 根因 -> 修复 -> 如何防止再次发生。比如“UART 偶发粘包，不是加 delay，而是用 DMA+IDLE、环形缓冲和长度/CRC 重新设计接收链路”。

7. 面试前 10 分钟速记表

通信协议

- **I2C**: SDA/SCL, 开漏 + 上拉, 地址 + ACK, START/STOP, 100k/400k/1M, 7 位常规地址 0x08-0x77。
- **SPI**: SCLK/MOSI/MISO/CS, CPOL+CPHA 四模式, 协议不规定地址格式, 也不强制 MSB first。
- **UART**: TX/RX, 无独立时钟, 起始位 + 数据位 + 校验 + 停止位; 项目常用 **DMA + IDLE** 不定长接收。
- **UART 错误**: ORE/FE/NE/PE 分别关注过载、帧、噪声、校验; RXNE/RXFNE 看“有字节”, IDLE 看“一段接收后出现空闲”。
- **CAN**: 控制器 + 收发器 + 双绞线, 主干两端 **120Ω**; 11/29 位 ID, Classical CAN 最多 8B, CAN FD 最多 64B。
- **RS485**: 差分电气层, 常配 UART 和 DE 控制; 长线/多节点注意终端、偏置和总线拓扑; 切换 DE 通常等 TC。
- **Modbus RTU**: Address + Function + Data + **CRC16**; CRC 低字节通常先发; 经典帧间隔至少 3.5 个字符时间。
- **Modbus**: 常用 01/02/03/04/05/06/0F/10 功能码; 地址 0 常用于广播; 异常帧功能码 = 原功能码 + 0x80。
- 40001 是参考编号, 不等于线路上直接发送十进制 40001; 注意厂家采用 0-based 还是 1-based。

STM32 / MCU

- **复位**: 向量表[0] -> **MSP**, 向量表[1] -> **Reset_Handler**; 启动代码初始化 .data/.bss 后进入 main。
- **GPIO**: 输入/输出/复用/模拟; **推挽**能主动拉高拉低, **开漏**只能主动拉低。
- **EXTI** 负责产生外部事件/**中断**请求; **NVIC** 负责 CPU 侧使能、优先级和响应。
- **NVIC**: 数值越小优先级越高; 抢占优先级决定能否嵌套, 子优先级决定同抢占级 pending 顺序。
- **中断**标志的清除方式不是统一写 0, 必须看目标系列 **Reference Manual**。
- **Timer**: 计数频率 = $\text{TimerClock}/(\text{PSC}+1)$, 更新频率再除 $(\text{ARR}+1)$; Input Capture 锁存 CNT, Output Compare 比较 CNT/CCR, PWM 是周期比较应用。
- **DMA**: CPU 配置, **DMA** 搬运, 完成/错误再通知 CPU; 循环模式适合连续流。
- **Bootloader**: 校验 APP -> 停外设/清**中断** -> **VTOR** -> **MSP** -> **Reset_Handler**。

C 语言

- **static**: 局部变量延长存储期; 文件作用域变量/函数变内部链接。
- **volatile**: 保留可观察访问; 不等于原子、锁、线程安全。
- 数组不是指针; 数组形参会调整为指针; 函数内 **sizeof(a)** 通常只能得到指针大小。
- `char *p = "abc"` 指向字面量, 不要修改; `char a[] = "abc"` 是可写数组副本。
- `T*` 的 `p+1` 前进一个 T; 指针运算应围绕同一数组对象。
- **sizeof** 看对象/类型字节数; **strlen** 数到 `\0` 前。**memcpy** 不允许重叠, **memmove** 允许。

- **realloc** 可能搬家，失败时旧指针仍有效；用临时指针接返回值。**memset** 按字节填充。
- **signed/unsigned** 混合比较要小心通常算术转换。
- **struct** 有 **padding**；**union** 共享存储；协议数据不要直接赌 ABI。

FreeRTOS

- **任务状态**：Running / Ready / Blocked / Suspended。
- 高优先级 Ready 任务可抢占；同优先级时间片通常 1 Tick，不是固定 1ms。
- **SysTick** 提供 Tick；**PendSV** 做上下文切换；**SVC** 常用于启动调度。
- **Queue** 传数据；**Binary Semaphore** 做同步；**Mutex** 做互斥并带**优先级继承**；Notification 最轻量。
- ISR 用 **FromISR** API；若唤醒更高优先级任务，退出**中断**前请求 yield。
- 任务栈要看高水位；大局部数组容易栈溢出；长临界区会拖慢被屏蔽的**中断**。

工程基础

- **map** 文件看符号地址和 **Flash/RAM** 占用；**链接脚本**决定各段放到哪块内存。
- 高优化会让源码调试不再逐行对应机器指令；Release 异常要警惕未定义行为、竞态和越界。
- 栈是 LIFO，队列是 FIFO；链表“插删 O(1)”以已经拿到目标位置为前提。